Homework 1

A. List allows duplicate elements and keeps insertion order. Set does not allow duplicates, and some implementations like HashSet do not guarantee order.

B. ArrayList is backed by a dynamic array, so random access is fast. LinkedList is better for frequent insertions and deletions in the middle.

C. Map stores key–value pairs. And keys must be unique, but values can be duplicated.

D. HashMap uses the key's hashCode() to decide which bucket to store data in. If two keys have the same hash, Java handles collision using linked lists or trees.

E. Hash collision happens when two different keys generate the same hash value. Java stores them in the same bucket and compares keys using equals().

F. Java Collections Framework provides data structures and utility methods for storing and manipulating data.

G. An immutable class cannot be changed after creation. For example, string is a common one.

H. Hashtable is thread–safe but slower. HashMap is not thread–safe. ConcurrentHashMap is thread–safe and performs better in multi–threaded.

I. String is immutable. StringBuilder is mutable and fast but not thread–safe. StringBuffer is mutable and thread–safe.

J. If two objects are equal according to equals(), they must return the same hashCode(). Otherwise collections like HashMap will not work correctly.

```
//List<String> list = new ArrayList<>();
//list.add("A");
//list.remove("A");
//
//Set<Integer> set = new HashSet<>();
//set.add(1);
//
//Queue<Integer> queue = new LinkedList<>();
//queue.offer(10);
//queue.poll();
//
//Map<Integer, String> map = new HashMap<>();
//map.put(1, "Jamie");
//map.get(1);
```

K. .

Homework 2

A. String is immutable. StringBuilder is faster for string modification. StringBuffer is thread-safe but slower.

B. Comparable defines natural ordering inside the class. Comparator is used when you want custom sorting logic outside the class.

C. Overloading means same method name with different parameters. Overriding means subclass provides a new implementation of parent method.

D. JVM runs Java bytecode. JRE contains JVM plus libraries to run Java applications. JDK includes JRE plus development tools like compiler.

E. Java has 8 primitive types: byte, short, int, long, float, double, char, and boolean.

F.Primitive types store actual values and reference types store addresses of objects.

G. JVM loads class files, converts bytecode into machine code, and manages memory and garbage collection.

H. JVM memory mainly includes Heap, Stack, Method Area, and PC Register. Heap stores objects while Stack stores method calls and local variables.

I. Garbage Collector automatically removes unused objects from memory by checking whether objects are still referenced.

J. Young generation stores new objects and old generation stores long-lived objects. Permanent generation used to store class metadata in older Java versions.

K. Common garbage collectors include Serial GC, Parallel GC, CMS, and G1 GC. G1 is commonly used because it balances performance and pause time.

Homework 3

A. public can be accessed everywhere. private only inside the class. protected allows subclass access. Default scope means package-private.

B. static belongs to the class instead of objects. All instances share the same static variable.

C. ClassLoader loads .class files into JVM memory dynamically when needed.

D. Checked exceptions must be handled during compile time. Unchecked exceptions happen during runtime.

E. finally always executes after try-catch. final prevents modification. finalize() was used before object destruction.

F. Try-with-resource automatically closes resources like files or database connections.

G. RuntimeException happens during program execution, usually caused by programming mistakes.

H. ClassNotFoundException happens when class loader cannot find a class. NoClassDefFoundError means class existed during compile time but missing at runtime.

I. It prevents resource leaks like unclosed files or database connections.

J. This error happens when JVM cannot allocate enough memory for objects.

K. Generics provide type safety and avoid manual casting.

L. Java removes generic type information during runtime. This is called type erasure.

M. extends is mainly for reading data safely. super is mainly for writing data safely.

N.

```
//      Optional<String> name = Optional.ofNullable(null);
//
//      System.out.println(name.orElse("Default"));
//
//      name.orElseThrow(() -> new RuntimeException("No value"));
```

O. OOP stands for Object-Oriented Programming. Main concepts are encapsulation, inheritance, polymorphism, and abstraction.

Homework 4

A. A functional interface has only one abstract method and is commonly used with lambda expressions.

B. Default methods allow interfaces to have method implementations.

C.Predicate returns boolean. Supplier provides data. Consumer consumes data without return. Function transforms data.

D.

```
//      Predicate<Integer> p = x -> x > 10;
//
//      Supplier<String> s = () -> "Hello";
//
//      Consumer<String> c = x -> System.out.println(x);
//
//      Function<Integer, Integer> f = x -> x * 2;
```

E.Method reference is a shorter way to write lambda expressions.

F.CompletableFuture is used for asynchronous programming and non-blocking tasks.

G. default keyword is used in interfaces for method implementation. Default scope means package-private access.

H.

```
//      List<Student> list = new ArrayList<>();
//
//      list.stream()
//              .filter(s -> s.getName().startsWith("A"))
//              .forEach(System.out::println);
//
//      int sum = list.stream()
//              .mapToInt(Student::getScore)
//              .sum();

```

I. Intermediate operations return another stream, like filter() and map(). Terminal operations produce final results, like collect() and forEach().

J.

```
//      String s = "jamie";
//
//      Map<Character, Long> result =
//              s.chars()
//                      .mapToObj(c -> (char)c)
//                      .collect(Collectors.groupingBy(
//                              c -> c,
//                              Collectors.counting()
//                      ));
//
```

K. map() transforms each element one-to-one. flatMap() flattens nested structures into a single stream.